

Counting 4-Cycles in Fully Dynamic Graphs: A Cyclic-Join Perspective

Jakob Fir

Faculty of Computer and Information Science
Ljubljana, Slovenia

Jure Snoj

Faculty of Computer and Information Science
Ljubljana, Slovenia

Janez Koprivec

Faculty of Computer and Information Science
Ljubljana, Slovenia

Gal Zmazek

Faculty of Computer and Information Science
Ljubljana, Slovenia

Abstract

Counting subgraphs in a fully dynamic graph is used in databases for incremental view maintenance (IVM) of cyclic joins. The worst-case update time is settled for two of the three patterns on at most four vertices: triangles at $O(\sqrt{m})$, 4-cliques at $O(m)$. For 4-cycles the previous upper bound was $O(m^{2/3})$ [6]; Assadi and Shah [3] recently improved this to $O(m^{2/3-\epsilon})$ for a constant $\epsilon > 0$ using fast matrix multiplication (FMM), a tool previously thought inapplicable to dynamic IVM. This essay re-presents their algorithm through the IVM-for-cyclic-joins lens and contributes (i) a closed-form algebraic certificate that the underlying constraint system is feasible at the conjectural $\omega = 2$ with explicit rational parameters, complementing the numerical verification of [3]; and (ii) an empirical study contrasting the warm-up algorithm of [3, §3] (with NumPy GEMM substituting for FMM) against a layer-aware Simple-Wedge baseline, which locates the input regime (bilateral hubs on both sides of the cyclic ordering) where the class-routing machinery actually pays off.

Keywords

fully dynamic algorithms, subgraph counting, 4-cycles, fast matrix multiplication, incremental view maintenance, cyclic joins

1 Introduction

Subgraph counting has many applications in computer science: in computational biology, for motif discovery in biological networks [10]; in social-network analysis, for clustering and community detection [14]; and in database systems, for recurring-subexpression evaluation in query processing [13]. The problem asks, given a host graph G and a small pattern graph H (typically a triangle, 4-cycle, or 4-clique), for the number of subgraphs of G isomorphic to H .

In database theory, subgraph counting underlies *incremental view maintenance* (IVM) for *cyclic joins*. A cyclic join is a query $R_1 \bowtie R_2 \bowtie \dots \bowtie R_k$ over k binary relations whose attribute schemas form a cycle: each relation shares one attribute with its predecessor and one with its successor, and the last relation closes back to the first. Such queries are a classical hard case for join evaluation [11, 12]. The four-way instance considered throughout this paper is

$$J = A(L_1, L_2) \bowtie B(L_2, L_3) \bowtie C(L_3, L_4) \bowtie D(L_4, L_1),$$

where each binary relation can be viewed as a bipartite edge set between two attribute domains, and $|J|$ equals the number of 4-cycles

in the resulting 4-layered graph (Section 4.1). IVM updates a query's answer after each tuple change; maintaining $|J|$ under updates is therefore equivalent to dynamic 4-cycle counting (Section 4).

In each of these settings the host graph evolves under edge insertions and deletions, and recomputing the count from scratch is infeasible at scale. This motivates the *fully dynamic* model of subgraph counting, in which the exact count must be reported after every update; we measure cost by the *worst-case update time*, the maximum work on any single update, which is strictly stronger than amortised time.

The complexity of dynamic subgraph counting on small patterns is well understood for two of the three patterns on at most four vertices, summarised in Table 1. Triangles have worst-case update time $\Theta(\sqrt{m})$ [7, 9] (the lower bound is conditional on the OMv conjecture); 4-cliques have complexity $\Theta(m)$ for combinatorial algorithms under the 4-clique conjecture [6]. The 4-cycle lies between these two extremes: Hanauer, Henzinger, and Hua [6] obtained $O(m^{2/3})$ by generalising the triangle bound's wedge-maintenance idea, with the only matching lower bound the $\Omega(m^{1/2-\gamma})$ inherited from triangles.

Assadi and Shah [3] prove that the $O(m^{2/3})$ upper bound is not tight. They give an algorithm with worst-case update time $O(m^{2/3-\epsilon})$ for a constant $\epsilon > 0$ depending on the matrix-multiplication exponent ω , the smallest constant for which two $n \times n$ matrices can be multiplied in $O(n^\omega)$ time. The current best is $\omega < 2.371339$ [1], yielding $\epsilon \approx 0.0098$; under the conjectural optimum $\omega = 2$ the bound improves to $\epsilon = 1/24$. The result is conceptually surprising: fast matrix multiplication (FMM) was previously believed incompatible with dynamic IVM (Section 2.4), and the Online Matrix-Vector multiplication (OMv) lower bound of [7] rules out only combinatorial improvements, leaving room for an algebraic speedup [3, §1].

Pattern	Upper bound	Lower bound
3-cycle (triangle)	$O(\sqrt{m})$ [9]	$\Omega(m^{1/2-\gamma})$ [7]
4-cycle	$O(m^{2/3-\epsilon})$ [3]	$\Omega(m^{1/2-\gamma})$ [7]
4-clique	$O(m)$ folklore [6]	$\Omega(m^{1-\gamma})$ [6]

Table 1: State of the art for fully dynamic subgraph counting on patterns of at most four vertices. Lower bounds are conditional (OMv for 3- and 4-cycles; combinatorial 4-clique conjecture for 4-cliques).

This essay traces the algorithm through the IVM-for-cyclic-joins lens and asks one structural question: where, exactly, does the sub- $m^{2/3}$ improvement live? Our answer points at a single pressure point. The restricted variant (Section 6) reduces every per-update query to a constant number of look-ups against stored matrix products, only one of which, the dense-block product $A^{L*} \cdot B_{i,DD}$ of Section 7, is a nontrivial use of FMM, and only that one has a cost governed by ω . The phase mechanism of the full algorithm (Section 8) promotes the restricted variant to a worst-case bound, but the per-update budget remains pinned to the same FMM step. Our two contributions probe this pressure point from complementary directions. *Theoretically*, we trace the parameter trade-off into a five-inequality constraint system and give a closed-form certificate that the system is feasible at the conjectural optimum $\omega = 2$ (Lemma 5); the certificate exposes *which* constraint binds tight (the rectangular FMM constraint, the only one of the five whose left-hand side depends on ω), thereby isolating FMM as the load-bearing ingredient. *Empirically*, we benchmark a faithful-in-shape implementation of the restricted variant against a layer-aware wedge baseline (Section 9); the comparison locates the input regime (bilateral hubs on both sides of the cyclic ordering) in which the warm-up’s class-routing actually fires, and confirms that on simpler inputs the layered baseline already captures most of the benefit. The two angles converge: the algebraic improvement of [3] lives at the rectangular FMM step, and the surrounding machinery exists to feed that step well-conditioned inputs.

2 Related work

Dynamic subgraph counting builds on three lines of work: static FMM-based subgraph algorithms, dynamic wedge-maintenance for triangles, and the 4-vertex result of Hanauer, Henzinger, and Hua. The following sections present each of these in turn, discuss why fast matrix multiplication was long thought to be incompatible with incremental view maintenance, and close with the Assadi-Shah algorithm.

2.1 Static algorithms for 4-cycles

The trivial static algorithm enumerates all *wedges* (length-2 paths $u-x-v$, i.e., pairs of edges sharing a common midpoint x) and, for each ordered pair (u, v) , counts the wedges between them; this gives $O(m^{3/2})$ time on graphs with m edges and is tight for combinatorial algorithms. Alon, Yuster, and Zwick [2] gave the first improvement to use fast matrix multiplication: their algorithm runs in time $O(m^{2\omega/(\omega+1)})$, which evaluates to roughly $O(m^{1.41})$ at the current ω and to $O(m^{4/3})$ at the conjectural optimum $\omega = 2$. The algorithm computes A^2 for the adjacency matrix A on the high-degree subgraph and combines it with a list-and-count step on the low-degree side; it was the first piece of evidence that FMM yields nontrivial speedups for 4-cycle counting in any setting. Crucially, this evidence was static: until 2025, no FMM-based speedup over the combinatorial bound was known in the dynamic regime.

2.2 Dynamic triangle counting and the OMv barrier

The triangle case is the template for this line of work. Kara, Ngo, Nikolic, Olteanu, and Zhang [9] maintain the triangle count in

worst-case update time $O(\sqrt{m})$ by storing, for each ordered pair of vertices (u, v) , the number of common neighbours $W[u, v] = |N(u) \cap N(v)|$. The number of triangles created by inserting an edge (u, v) is exactly $W[u, v]$; updating W after the insertion costs $O(\deg(u))$ for the row indexed by u and $O(\deg(v))$ for the row indexed by v , and a high/low-degree partition (with threshold $\sqrt{m}$) caps the relevant degrees and gives the $\sqrt{m}$ bound. The OMv conjecture of Henzinger, Krinninger, Nanongkai, and Saranurak [7] implies that $\Omega(m^{1/2-\gamma})$ is the right answer; this lower bound holds even against algorithms that invoke fast matrix multiplication, because the OMv reduction is combinatorially robust. The same lower bound is inherited by 4-cycle counting via a straightforward reduction; [8] additionally show that the bound $\Omega(m^{1/2-\gamma})$ already holds for 4-cycle counting on random graphs, ruling out average-case improvements through that channel. Earlier work of [4, 5] maintained 3- and 4-vertex subgraph counts in $O(h)$ amortised update time, where h is the h -index of the graph; this is incomparable with bounds expressed purely in terms of m and is not a direct predecessor of the worst-case $O(\sqrt{m}) / O(m^{2/3})$ line of work, but it is the closest relative on the dynamic side.

2.3 The $O(m^{2/3})$ algorithm of Hanauer, Henzinger, and Hua

Hanauer, Henzinger, and Hua [6] extended the high/low-degree wedge-maintenance idea from triangles to all 4-vertex patterns. For 4-cycles, they partition the vertex set V into a *low-degree* class $V_L = \{v : \deg(v) \leq m^{1/3}\}$ and a *high-degree* class $V_H = V \setminus V_L$. The handshaking bound $\sum_v \deg(v) = 2m$ caps $|V_H|$ at $\frac{2m}{m^{1/3}} = O(m^{2/3})$, which is the load-bearing inequality of the analysis.

The algorithm maintains two indexed counts:

- $W_{lo}[u, v]$, the number of wedges $u-x-v$ with low-degree midpoint $x \in V_L$, stored for every pair of endpoints (u, v) ;
- $P_{lo}[u, v]$, the number of 3-paths $u-x-y-v$ with two low-degree midpoints $x, y \in V_L$, stored for every pair of endpoints.

On an update incident to u , the affected entries of W_{lo} are those of the form $W_{lo}[w, v]$ for a low-degree $w \in N(u) \cap V_L$ and a neighbour v of w ; iterating the inner pair costs at most $|N(u)| \cdot \max_{w \in V_L} |N(w)| \leq n \cdot m^{1/3}$ in the trivial reading, but only the low-degree neighbours of w contribute new wedges, and there are at most $m^{1/3}$ of them, so the work reduces to $|N(u)| \cdot m^{1/3} \leq m^{1/3} \cdot m^{1/3} = m^{2/3}$ when $u \in V_L$ and to $|N(u)| \cdot m^{1/3}$ in general. A symmetric analysis on P_{lo} keeps it within the same $O(m^{2/3})$ budget. Wedges through high-degree midpoints are maintained only between pairs of high-degree endpoints; there are $|V_H|^2 = O(m^{4/3})$ such pairs, but the per-update changes are confined to the at most $|V_H| = O(m^{2/3})$ pairs that share an endpoint with the updated edge.

Queries case-split on whether each midpoint of the counted 3-path is low or high. The bottleneck case, with two high-degree midpoints and at least one low-degree endpoint u , requires iterating over $N(u)$, which has size at most $m^{1/3}$, and combining stored counts; the inner step iterates over high-degree vertices, of which there are $O(m^{2/3})$. Either iteration alone gives a query cost of $O(m^{2/3})$, matching the maintenance budget.

This decomposition suggests that $O(m^{2/3})$ should be tight: 4-cycles would sit exactly at the geometric mean of triangles and

4-cliques, and any further improvement would require a fundamentally different technique. Assadi-Shah [3] show that this reading is mistaken: algebraic FMM techniques can push the upper bound below $m^{2/3}$.

2.4 Why FMM was thought not to help in IVM

Static FMM-based subgraph counting [2] requires both factors of the product, typically two slices of the adjacency matrix, to be assembled before the multiplication is invoked. In the fully dynamic regime the matrices are themselves the moving objects: each edge update changes a single entry, and waiting until enough updates accumulate to make a batched FMM call worthwhile only gives an *amortised* bound, not a worst-case one. The prevailing intuition was therefore that any FMM-based speedup must be amortised, and could not deliver a worst-case improvement of the kind we discuss [3, §1].

Assadi and Shah [3] address this intuition by introducing *phases*: contiguous blocks of $m^{1-\delta}$ consecutive updates, for a constant $\delta > 0$. During a phase, queries are answered by a slower routine that uses precomputed products from the *previous* phase together with on-the-fly computation over the in-progress phase; in parallel, the FMM products needed for the *current* phase are computed incrementally, with the per-update share of the precomputation budgeted at $O(m^{2/3-\epsilon})$ work. By the end of the phase, the precomputed products are ready, and the next phase reuses them as its previous-phase data. Because both the per-update share of precomputation and the slower routine respect an $O(m^{2/3-\epsilon})$ ceiling on every individual update, not merely on average, the bound is genuinely worst-case (Section 3.2) rather than amortised.

2.5 Overview of the Assadi-Shah algorithm

The full Assadi-Shah algorithm runs on a 4-layered graph, groups the vertices in each layer into many degree classes (high, medium, low in the outer layers; dense, sparse in the inner layers), and precomputes one rectangular FMM product per pair of compatible classes. A query then composes a constant number of stored entries plus a small-degree iteration. The ϵ in the exponent is the maximum value for which the system of constraints (one per class pair, expressing that both setup cost and per-query cost stay within the budget) is feasible at the prevailing rectangular FMM exponents. The argument also requires two simplifying assumptions which the paper later discharges: that every layer contains exactly $m^{2/3}$ vertices (no *tiny* vertices), and that vertices do not migrate between degree classes during a phase.

In Section 4 we present the reduction from general graphs to 4-layered graphs; Section 5 develops the simple $O(n)$ baseline, which also serves as the algorithm we benchmark; and Sections 6–8 cover the restricted variant of the Assadi-Shah algorithm with two frozen relations, its rectangular FMM step in full, and a closed-form verification of the constraint system. The technical machinery used by [3] to discharge their two simplifying assumptions (handling layers with very few vertices, and tracking vertices that change degree class during a phase) is out of scope here.

3 Preliminaries

3.1 Graphs and subgraph notation

Let $G = (V, E)$ be a simple, undirected, unweighted graph with no self-loops or parallel edges. We write $n = |V|$ and $m = |E|$. For a vertex $v \in V$, $\deg(v)$ denotes its degree and $N(v) = \{u : \{u, v\} \in E\}$ its neighborhood.

A k -*path* is a sequence of $k + 1$ distinct vertices $(v_0, v_1, \dots, v_k)$ with $\{v_i, v_{i+1}\} \in E$ for all $0 \leq i < k$. A k -*cycle* is a sequence $(v_0, \dots, v_{k-1})$ of k distinct vertices with $\{v_i, v_{i+1 \bmod k}\} \in E$ for all $0 \leq i < k$. A *wedge* is a 2-path (u, x, v) , equivalently a pair of edges $\{u, x\}, \{x, v\} \in E$ sharing the common midpoint x with distinct endpoints $u \neq v$. We write $W[u, v]$ for the number of wedges between u and v .

A *4-layered graph* is a graph whose vertex set partitions as $V = L_1 \cup L_2 \cup L_3 \cup L_4$, with each L_i an independent set and edges only between consecutive layers L_i, L_{i+1} (and between L_4 and L_1 , closing the cycle). The four edge sets are encoded by biadjacency matrices $A \subseteq L_1 \times L_2, B \subseteq L_2 \times L_3, C \subseteq L_3 \times L_4$, and $D \subseteq L_4 \times L_1$. A *layered k -path* (resp. *layered k -cycle*) is a k -path (resp. k -cycle) with one vertex in each consecutive layer. We index layers modulo 4 throughout.

3.2 Dynamic graph model

The graph evolves under a sequence of updates $u_1, u_2, \dots$, where each u_i is either $\text{INSERT}(e)$ or $\text{DELETE}(e)$ for an edge e on a fixed vertex set. Let G_i be the graph after the i -th update, with G_0 the empty graph. The model is *fully dynamic* because both insertions and deletions are permitted. Immediately after each update, the algorithm must output the exact number of 4-cycles in G_i .

The complexity measure is the *worst-case update time*: the maximum time spent processing any single update, taken over every update index and every legal sequence. This is strictly stronger than *amortized* time, which bounds only the average over a sequence; the headline bound of [3] is worst-case.

3.3 Fast matrix multiplication

Let ω denote the matrix-multiplication exponent: two $n \times n$ matrices can be multiplied in $O(n^\omega)$ arithmetic operations. The current record is $\omega < 2.371339$ [1], improving on Strassen's $\omega < 3$ [15]; whether $\omega = 2$ remains open. For rectangular products, $\omega(a, b, c)$ denotes the exponent of multiplying an $n^a \times n^b$ matrix by an $n^b \times n^c$ matrix in $n^{\omega(a, b, c)}$ time; the inner dimension n^b is shared by the two factors, while n^a and n^c are the outer dimensions of the left and right factors respectively, so $\omega(1, 1, 1) = \omega$. Improved bounds across the asymmetric range are also in [1]. An algorithm is *combinatorial* if it does not exploit Strassen-style identities; combinatorial algorithms therefore do not benefit from $\omega < 3$.

The use of FMM in subgraph counting rests on the identification of biadjacency-matrix products with path counts: if X is the biadjacency matrix between layers L_i and L_{i+1} and Y is the biadjacency matrix between L_{i+1} and L_{i+2} , then $(XY)_{u, w}$ equals the number of 2-paths from $u \in L_i$ to $w \in L_{i+2}$ through L_{i+1} . Iterating the product, $(XYZ)_{u, t}$ counts the 3-paths from u to t through three consecutive layers, which is exactly the quantity that drives 4-cycle queries in the layered setting (Theorem 1). Restricting X, Y, Z to specific

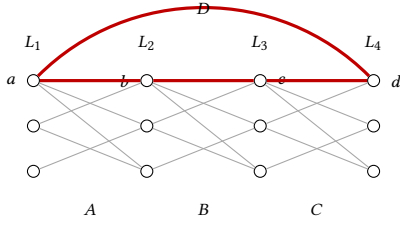


Figure 1: A 4-layered graph encoding the cyclic join $A \bowtie B \bowtie C \bowtie D$. Each binary relation is a bipartite edge set between two attribute domains. The highlighted layered 4-cycle (a, b, c, d) corresponds to a tuple in J . Adapted from [3], Fig. 1.

degree classes yields rectangular factors whose products can be computed with the rectangular FMM bound $\omega(a, b, c)$.

3.4 The OMv conjecture

The *Online Matrix-Vector multiplication (OMv) conjecture* of [7] concerns the following problem: an algorithm preprocesses an $n \times n$ Boolean matrix M in polynomial time, then receives n Boolean vectors $v_1, \dots, v_n$ one at a time and must output Mv_i before seeing v_{i+1} . The conjecture asserts that no algorithm solves the entire instance in total time $O(n^{3-\gamma})$ for any constant $\gamma > 0$. The $\Omega(m^{1/2-\gamma})$ lower bounds for dynamic triangle and 4-cycle counting (Section 2.2) are reductions from OMv; we cite the conjecture only to motivate the lower bound and do not invoke it in any proof.

4 The cyclic-join equivalence

4.1 Joins as layered graphs

The motivating object behind 4-cycle counting on a 4-layered graph is the cyclic four-way join

$$J = A(L_1, L_2) \bowtie B(L_2, L_3) \bowtie C(L_3, L_4) \bowtie D(L_4, L_1),$$

familiar in database theory as a fundamental hard case for join evaluation [11, 12]. The four binary relations A, B, C, D over the attribute domains $L_1, \dots, L_4$ are exactly the biadjacency matrices of Section 3.1: a tuple in the relation between L_i and L_{i+1} corresponds to an edge between the two consecutive layers in $G' = (L_1 \cup L_2 \cup L_3 \cup L_4, E')$. A satisfying assignment $(a, b, c, d) \in L_1 \times L_2 \times L_3 \times L_4$ of J is precisely a closed sequence $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$ that visits one vertex per layer, that is, a layered 4-cycle in G' . The size of the join therefore equals the number of layered 4-cycles in G' . Figure 1 illustrates the correspondence on a small instance. Maintaining $|J|$ under tuple updates is then exactly the incremental view-maintenance problem for this cyclic join, which is the database-theoretic reading of our counting task.

4.2 Lifting a general graph to a 4-layered graph

From a general graph $G = (V, E)$, build a 4-layered graph $G' = (V', E')$ by replicating V in each layer: $V' = V \times \{1, 2, 3, 4\}$, with the i -th copy of V playing the role of L_i . Every edge $\{u, v\} \in E$ is lifted into all four biadjacency matrices: each of A, B, C, D records the pair (u, v) (equivalently, each layer pair $L_i \rightarrow L_{(i \bmod 4) + 1}$ gains the two directed edges (u_i, v_{i+1}) and (v_i, u_{i+1}) that encode the undirected

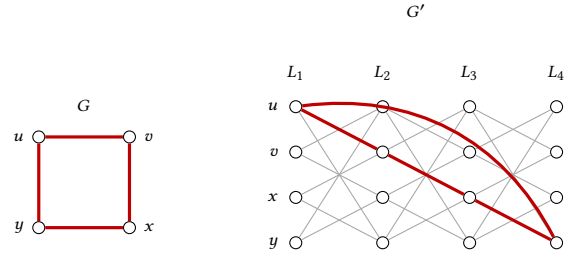


Figure 2: Lifting a general graph G (left) to a 4-layered graph G' (right). Every vertex of V is replicated in each layer $L_1, \dots, L_4$; gray edges schematically depict the lifted copies between consecutive layers. The 4-cycle u, v, x, y in G corresponds to the highlighted layered 4-cycle $(u, 1) \rightarrow (v, 2) \rightarrow (x, 3) \rightarrow (y, 4) \rightarrow (u, 1)$, with the closing $L_4 \rightarrow L_1$ edge drawn as a curved arc. For legibility, only the highlighted cycle is shown in full; remaining lifted edges are partial.

edge between adjacent-layer copies). The construction is depicted in Figure 2.

Crucially, a single update (u, v) in G becomes four updates in G' , one per matrix, and the order in which these are applied is critical for the proof of Theorem 1. We adopt the following *update protocol* [3]:

Insertion. Insert (u, v) first into D , then into C , then into B , and finally into A .

Deletion. Delete (u, v) from A first, then from B , then from C , and last from D .

Query. Read off the change in the layered 4-cycle count immediately after the D -update, between the D step and the next of A, B, C .

In both cases, the protocol guarantees that the lifted edge (u, v) is *absent* from A, B , and C at the moment of query: on an insertion the edge has not yet propagated past D , and on a deletion it has already been removed from A, B, C before D . This asymmetry between D and the other three matrices is exactly what forces every length-3 walk counted by the algorithm to be a length-3 *path* in the underlying graph; without it, a walk could revisit u or v via the freshly written edge and the count would over-report.

4.3 Equivalence theorem

Theorem 1 (General-to-layered equivalence). *Let $\{u, v\}$ be the edge currently being updated in G , and let G' be the lifted graph of Section 4.2 immediately after the D -step of the protocol (and before any subsequent A, B, C step). At this moment, the number of length-3 walks in G' from $(u, 1) \in L_1$ to $(v, 4) \in L_4$ equals the number of length-3 paths from u to v in the current G .*

PROOF SKETCH. We follow [3]. The forward direction is by construction: each length-3 path u, x, y, v in G lifts to the walk $(u, 1), (x, 2), (y, 3), (v, 4)$ in G' , using one edge of A, B, C each. For the reverse direction, fix such a walk and write its underlying sequence as u, x, y, v . Simplicity of G (no self-loops to lift) rules out $x \in \{u, y\}$ and $y = v$. The protocol rules out $x = v$ and $y = u$: each would require the edge $\{u, v\}$ to be present in A or C at query time, but the D -first ordering

keeps (u, v) absent from A, B, C throughout the query window. The four vertices are therefore distinct, and the bijection follows. $\square$

4.4 Consequence

Theorem 1 reduces the dynamic 4-cycle counting problem on a general graph to dynamically counting layered 4-cycles on a 4-layered graph: the number of 4-cycles through a new edge (u, v) is exactly the layered 3-walk count delivered by the lifted instance under the protocol of Section 4.2, and the lift incurs only a constant blow-up in the number of layered updates per general-graph update. The restricted algorithm of Section 6 therefore works directly on a 4-layered graph with biadjacency matrices A, B, C, D . The simple $O(n)$ baseline of Section 5 is presented in the original general-graph setting and uses the same query-then-update timing idea adapted to its single wedge matrix.

5 A simple $O(n)$ algorithm

5.1 The wedge-counting idea

The simple algorithm of this section operates directly on a general graph $G = (V, E)$ on n vertices, without invoking the layered reduction of Section 4. The starting observation is that an edge update only changes the 4-cycle count through cycles *containing the updated edge*: if the count before an insertion of (u, v) is C_{old} , the count afterwards is $C_{\text{old}} + \Delta$, where Δ is the number of 4-cycles in the new graph that use the edge (u, v) . Each such 4-cycle is the closure $u \rightarrow w \rightarrow x \rightarrow v \rightarrow u$ of a length-3 path from u to v in the graph $G \setminus \{(u, v)\}$ (with the edge removed), so

$$\Delta = \#\{\text{length-3 paths from } u \text{ to } v \text{ in } G\}.$$

A length-3 path decomposes as a first edge $u \rightarrow w$ with $w \in N(u)$ followed by a length-2 path (a *wedge*) from w to v . Let $W[w, v]$ denote the number of wedges between w and v in G . Then

$$\Delta = \sum_{w \in N(u)} W[w, v]. \quad (1)$$

In principle the right-hand side counts length-3 *walks* u, w, x, v rather than paths; the order-of-operations rule of Section 5.2 guarantees that at query time the edge (u, v) is absent from W , which forces every counted walk to be a path with four distinct vertices and makes identity (1) hold without any correction term. The algorithm therefore reduces 4-cycle maintenance to maintaining the wedge-count matrix W .

5.2 The algorithm

Algorithm 1 maintains a global counter C for the current 4-cycle count alongside the wedge matrix W . Each update performs two distinct kinds of work: a *query* that uses identity (1) to compute the cycles through the updated edge, and a *maintenance* step that adjusts W to reflect the new edge set. Both steps touch only the rows and columns indexed by neighbours of u and v .

The order in which these two steps are performed is critical, and is the general-graph specialisation of the update protocol of Section 4.2: in both settings the new edge must be *absent* from the relevant data structure at the moment of query. For correctness, W must not contain any wedge that uses the edge (u, v) at the moment the query is evaluated; otherwise the expression $\sum_{w \in N(u)} W[w, v]$

Algorithm 1 Maintain 4-cycle count via wedge matrix W .

```

1: procedure INSERT( $u, v$ )
2:    $\Delta \leftarrow \sum_{w \in N(u)} W[w, v]$  ▷ query
3:    $C \leftarrow C + \Delta$ 
4:   for  $w \in N(u)$  do  $W[w, v] += 1$ ;  $W[v, w] += 1$ 
5:   end for
6:   for  $w \in N(v)$  do  $W[w, u] += 1$ ;  $W[u, w] += 1$ 
7:   end for
8:   add  $(u, v)$  to  $E$  ▷ after query, to keep  $W$  free of  $(u, v)$ 
   during query
9: end procedure
10: procedure DELETE( $u, v$ )
11:   remove  $(u, v)$  from  $E$ 
12:   for  $w \in N(u)$  do  $W[w, v] -= 1$ ;  $W[v, w] -= 1$ 
13:   end for
14:   for  $w \in N(v)$  do  $W[w, u] -= 1$ ;  $W[u, w] -= 1$ 
15:   end for
16:    $\Delta \leftarrow \sum_{w \in N(u)} W[w, v]$  ▷ query
17:    $C \leftarrow C - \Delta$ 
18: end procedure

```

would count walks that revisit u or v , breaking identity (1). On an INSERT, the edge (u, v) is not yet in E when the procedure starts, so the query is evaluated *first*, and only afterwards is W updated to record the wedges newly created through the inserted edge. On a DELETE, the edge (u, v) is still in E at the start, so W is updated *first*, removing the wedges that used (u, v) , and the query is evaluated only once those wedges are gone. In both cases, W at query time encodes only wedges of the graph $G \setminus \{(u, v)\}$, which is what identity (1) requires.

5.3 Correctness and complexity

Lemma 2 (Correctness and complexity of Algorithm 1). *Algorithm 1 maintains the exact 4-cycle count of any fully dynamic graph in worst-case update time $O(n)$, where $n = |V|$ is the (fixed) vertex count.*

PROOF SKETCH. *Complexity.* Each update touches at most $|N(u)| + |N(v)| \leq 2n$ entries of W ; the query is an inner product of length $|N(u)|$. Total: $O(n)$ per update.

Correctness. The query counts *walks* u, w, x, v in W . Simplicity of G rules out the collisions $w = u$, $x = w$, $x = v$; the timing rule of Section 5.2 rules out $w = v$ and $x = u$ (each would need the absent edge (u, v) in W). So every counted walk is a length-3 path, and each such path is counted exactly once. Full case analysis in [3, App. A]. $\square$

5.4 Limitations of the $O(n)$ bound

The $O(n)$ bound is tight in the worst case (a high-degree update touches $\Theta(n)$ entries of W) and matches the natural m -dependent target only when $m = \Theta(n)$; on denser graphs the algorithm does work unrelated to the current edge count. A second, practical, drawback is the n^2 -entry memory footprint: ≈ 16 MB at $n = 2000$ but ≈ 4 GB at $n = 32,000$ regardless of edge density.

These two shortcomings motivate algorithms whose update time depends only on m . The first such bound was the $O(m^{2/3})$ algorithm of [6], which extends the wedge-counting idea above with a degree-based vertex partition. Section 6 develops the warm-up of [3], whose update time is $O(m^{2/3-\varepsilon_1})$ under simplifying assumptions, and which is the building block for the full $O(m^{2/3-\varepsilon})$ bound.

6 A restricted variant: fixed A and C

This section presents the restricted variant of the Assadi-Shah algorithm: a stripped-down version that already contains the central trick (precomputed FMM products combined with class-restricted iteration) but in which two of the four biadjacency matrices are frozen and two simplifying assumptions are imposed. The de-assumption machinery, occupying §6 and §7 of [3], is left out of scope; we sketch the move from the restricted variant to the full algorithm in Section 8.

6.1 Setup and assumptions

In the restricted variant, edge updates affect only B and D , while A and C are fixed in advance. By symmetry it suffices to handle the query case where the latest update is to D and we count new 4-cycles by counting 3-paths from $u \in L_1$ to $v \in L_4$ through A, B, C .

Two simplifying assumptions support the analysis: (1) each layer has at most $m^{2/3+2\varepsilon}$ vertices (for a small constant $\varepsilon > 0$ fixed below), ruling out pathologically thin edge sets; (2) no vertex changes degree class during the algorithm. Both are discharged in [3, §§6–7] and are out of scope here. The target update time is $O(m^{2/3-\varepsilon_1})$ for a constant $\varepsilon_1 > 0$, which is also what the full algorithm needs when this restricted variant is invoked as a subroutine.

6.2 Vertex degree classes

Outer-layer vertices (in L_1 and L_4) are partitioned by degree in the fixed matrices A and C respectively into three classes: *High* (degree $\geq m^{2/3-\varepsilon_1}$), *Low* (degree $\leq m^{1/3+\varepsilon_1}$), and *Medium* in between. A small overlap at the H/M boundary keeps the cut on a single threshold; details in [3, §3.1]. We write A^{H*}, A^{M*}, A^{L*} for the row-restrictions of A by class, and C^{*H}, C^{*M}, C^{*L} for the column-restrictions of C .

Inner-layer vertices (in L_2, L_3) are classified per chunk (§6.3): inside chunk B_i , a vertex is *Dense* if its degree within that chunk exceeds $m^{1/3-\varepsilon_2}$, and *Sparse* otherwise. The four chunk-submatrices $B_{i,DD}, B_{i,DS}, B_{i,SD}, B_{i,SS}$ are obtained by restricting both endpoints by class. The Medium class (absent from the earlier $O(m^{2/3})$ algorithm of [6]) is what makes room for the rectangular FMM call: it gives the dense-block product $A^{L*} \cdot B_{i,DD}$ the asymmetric dimensions that current rectangular bounds can exploit.

6.3 Chunks and lazy maintenance

B -updates are grouped into *chunks* of size $m^{2/3-\varepsilon_1}$; we write $B_{<i} := \sum_{j<i} B_j$ for the aggregate of all completed chunks. Cheap bookkeeping is maintained eagerly. The expensive matrix products are not per-update affordable, so they are recomputed once per chunk: the products needed for $B_{<i+1}$ are built during the $m^{2/3-\varepsilon_1}$ updates of chunk B_{i+1} itself, with a total budget of $O(m^{4/3-2\varepsilon_1})$, i.e., $O(m^{2/3-\varepsilon_1})$ per update.

While inside chunk B_{i+1} , queries combine the previous-chunk products with a *lazy* edge-by-edge scan of the active chunks B_i, B_{i+1} ,

Class	Stored products
High in L_1, L_4	$A^{H*} \cdot B_{<i}; A^{H*} \cdot B_{<i} \cdot C^{*H}; B_{<i} \cdot C^{*H}$
Medium in L_1, L_4	$A^{M*} \cdot B_{<i}; B_{<i} \cdot C^{*M}$
Low in L_1, L_4	$A^{L*} \cdot B_{<i,DD}, A^{L*} \cdot B_{<i,SS}, A^{L*} \cdot B_{<i,SD},$ and three symmetric products with C^{*L}

Table 2: Data structures maintained by the restricted algorithm. Adapted from [3], Table 1.

costing $O(1)$ per edge. Edges deleted in a later chunk than they were inserted are recorded as negative entries [3, §3.3], so the chunk-aggregate stays correct without retroactive edits.

6.4 Data structures

Table 2 lists the matrix products maintained across the three outer-layer classes, all evaluated against the chunk-aggregate $B_{<i}$. Each product encodes a 2- or 3-hop count between the indicated class pair; in database terms, this is a select-project-aggregate over the join $A \bowtie B$ (or $B \bowtie C$) restricted to that class. The High-High triple product $A^{H*} \cdot B_{<i} \cdot C^{*H}$ stores 3-path counts between any two High endpoints, so any HH query is answered in $O(1)$. Medium classes get only one-sided products and pay an iteration cost on the third hop. Low classes need a finer Dense/Sparse decomposition of $B_{<i}$ so that the dense-block product $A^{L*} \cdot B_{<i,DD}$, the only one of these products that actually requires FMM, can be amortized once per chunk.

Lemma 3 (Update-time accounting for Table 2). *The worst-case time to update all data structures in Table 2 is $O(m^{2/3-\varepsilon_1})$ per update, with $O(m^{4/3-2\varepsilon_1})$ amortized over chunk transitions.*

6.5 Answering queries

A query (u, v) with $u \in L_1, v \in L_4$ is answered by case analysis on the classes of u and v [3, Lemma 3.8]:

Both High. $O(1)$: read the precomputed triple product.

Mixed. $O(m^{2/3-\varepsilon_1})$: iterate the lower-class endpoint’s row (at most $m^{2/3-\varepsilon_1}$ entries) and look up two-hop counts.

Both Low. $O(m^{1/3+\varepsilon_1})$: iterate the low-degree neighbourhood ($2m^{1/3+\varepsilon_1}$ entries) and combine the four DD/SS/SD/DS contributions.

Plus a lazy edge-scan of the active chunks B_i, B_{i+1} , costing another $O(m^{2/3-\varepsilon_1})$. Combining these bounds with Lemma 3 yields the main statement of the warm-up: under our two assumptions, 4-cycles in a fully dynamic 4-layered graph are counted in worst-case time $O(m^{2/3-\varepsilon_1})$.

7 The FMM step and the constraint system

This section opens up the single pressure point of the restricted algorithm: the dense-block product $A^{L*} \cdot B_{i,DD}$, evaluated once per chunk transition. Every other ingredient of the warm-up respects the per-chunk budget by counting (the eager-update bookkeeping, the lazy in-chunk evaluation, the threshold-class iterations), but this product is the only nontrivial use of fast matrix multiplication in the algorithm, and the only step whose cost is governed by ω . We bound that cost (Section 7.1), set up the constraint system the cost

must satisfy together with three threshold orderings (Section 7.2), and exhibit a closed-form certificate of feasibility at $\omega = 2$ that pins down which constraints bind (Section 7.3).

7.1 Cost of the rectangular FMM step

Theorem 4 (Cost of the rectangular FMM step [3, Claim 3.6]). *The matrix product $A^{L*} B_{i,DD}$ can be computed in time $m^{\omega(2/3+2\varepsilon, 1/3-\varepsilon_1+\varepsilon_2, 1/3-\varepsilon_1+\varepsilon_2)}$.*

PROOF. We bound the effective dimensions of the two factors. The matrix A^{L*} has at most $|L_1| \leq n \leq m^{2/3+2\varepsilon}$ rows by Assumption 1, and we restrict its columns to indices in L_2 that are dense within chunk B_i . By definition, a Dense vertex of L_2 has degree at least $m^{1/3-\varepsilon_2}$ in B_i , so the total number of dense vertices in L_2 is at most $|B_i|/m^{1/3-\varepsilon_2} = m^{(2/3-\varepsilon_1)-(1/3-\varepsilon_2)} = m^{1/3-\varepsilon_1+\varepsilon_2}$. Hence the relevant dimensions of A^{L*} are $m^{2/3+2\varepsilon} \times m^{1/3-\varepsilon_1+\varepsilon_2}$. The factor $B_{i,DD}$ is the restriction of B_i to dense rows and dense columns, so by the same accounting it has dimensions $m^{1/3-\varepsilon_1+\varepsilon_2} \times m^{1/3-\varepsilon_1+\varepsilon_2}$. The product is therefore a rectangular matrix multiplication of shape $(2/3 + 2\varepsilon, 1/3 - \varepsilon_1 + \varepsilon_2, 1/3 - \varepsilon_1 + \varepsilon_2)$, which can be performed in time $m^{\omega(2/3+2\varepsilon, 1/3-\varepsilon_1+\varepsilon_2, 1/3-\varepsilon_1+\varepsilon_2)}$ by definition of the rectangular FMM exponent [1]. $\square$

7.2 The constraint system

Charging the cost of Theorem 4 against the per-chunk budget $m^{4/3-2\varepsilon_1}$ from Lemma 3 yields the inequality

$$\omega(2/3 + 2\varepsilon, 1/3 - \varepsilon_1 + \varepsilon_2, 1/3 - \varepsilon_1 + \varepsilon_2) \leq 4/3 - 2\varepsilon_1, \quad (2)$$

the FMM constraint for the Low-Dense block. The High-High product gives a symmetric FMM constraint, and three further inequalities (threshold orderings on $\varepsilon, \varepsilon_1, \varepsilon_2$ that capture the overlap conditions on the class boundaries) complete the system:

$$\omega(2/3 + 2\varepsilon, 1/3 - \varepsilon_1 + \varepsilon_2, 1/3 - \varepsilon_1 + \varepsilon_2) \leq 4/3 - 2\varepsilon_1,$$

$$\omega(1/3 + \varepsilon_1, 2/3 - \varepsilon_1, 1/3 + \varepsilon_1) \leq 4/3 - 2\varepsilon_1,$$

$$3\varepsilon_1 + 2\varepsilon \leq \varepsilon_2, \quad \varepsilon_1 \leq 1/6, \quad \varepsilon_1 - \varepsilon_2 \leq 1/3.$$

The first two are the FMM constraints, the only inequalities whose left-hand side depends on ω ; the last three are simple algebraic relations among the thresholds (Equations (5), (2), (6), (7), (8) of [3, §3.4], in that order). With current rectangular FMM bounds [1] the system is feasible at $\varepsilon_1 \approx 0.0420$, $\varepsilon_2 \approx 0.1457$, $\varepsilon \approx 0.0098$ [3, App. B]. The next subsection gives a clean closed-form solution at the conjectural optimum $\omega = 2$.

7.3 Closed-form feasibility at $\omega = 2$

Lemma 5 (Closed-form feasibility at $\omega = 2$). *Under the hypothetical $\omega = 2$, the constraint system above admits the solution $\varepsilon_1 = 1/24$, $\varepsilon_2 = 5/24$, $\varepsilon = 1/24$.*

PROOF. At $\omega = 2$, square matrix multiplication runs in input-size time, and the rectangular exponent collapses to $\omega(a, b, c) = \max(a + b, b + c, a + c)$, the cost of merely reading the largest pair of factors. We verify each constraint in turn.

Threshold ordering. $\varepsilon_1 \leq 1/6$ (constraint (7) of [3]): $1/24 < 4/24 = 1/6$. *Threshold ordering.* $\varepsilon_1 - \varepsilon_2 \leq 1/3$ (constraint (8) of [3]): $1/24 - 5/24 = -4/24 < 1/3$. *Sparse-Sparse / Sparse-Dense iteration.* $3\varepsilon_1 + 2\varepsilon \leq \varepsilon_2$ (constraint (6) of [3]): $3 \cdot \frac{1}{24} + 2 \cdot \frac{1}{24} = \frac{5}{24} = \varepsilon_2$, so the constraint holds with equality. *Low-Dense FMM*, our Eq. (2) (constraint (5)

of [3]): substitute $a = 2/3 + 2\varepsilon = 2/3 + 1/12 = 3/4$ and $b = c = 1/3 - \varepsilon_1 + \varepsilon_2 = 1/3 + 4/24 = 1/2$. Then

$$\omega(3/4, 1/2, 1/2) = \max\left(\frac{3}{4} + \frac{1}{2}, \frac{1}{2} + \frac{1}{2}, \frac{3}{4} + \frac{1}{2}\right) = \frac{5}{4}.$$

The right-hand side is $4/3 - 2\varepsilon_1 = 4/3 - 1/12 = 16/12 - 1/12 = 15/12 = 5/4$, so the constraint is satisfied with equality. *High-High product* (constraint (2) of [3]), $\omega(1/3 + \varepsilon_1, 2/3 - \varepsilon_1, 1/3 + \varepsilon_1) \leq 4/3 - 2\varepsilon_1$: the dimensions evaluate to $a = c = 9/24 = 3/8$ and $b = 15/24 = 5/8$. At $\omega = 2$,

$$\omega(3/8, 5/8, 3/8) = \max\left(\frac{3}{8} + \frac{5}{8}, \frac{5}{8} + \frac{3}{8}, \frac{3}{8} + \frac{3}{8}\right) = 1 \leq 5/4 = 4/3 - 2\varepsilon_1,$$

verified by the same method as our Eq. (2). All five constraints are satisfied, so the proposed assignment is feasible. $\square$

The verification highlights why FMM is essential. At $\omega = 2$ with this assignment, two constraints bind tightly: the Low-Dense FMM constraint (our Eq. (2), equivalently constraint (5) of [3]) and the SS/SD threshold (constraint (6)); the remaining three (the High-High FMM and constraints (7), (8)) have slack. Of these two tight constraints, the Low-Dense FMM is the load-bearing one: the only ω -dependent constraint of the system that binds, and the constraint that would force $\varepsilon_1 = 0$ if FMM were unavailable. The other tight constraint is a purely algebraic threshold-overlap condition that holds independently of ω . Appendix B of [3] runs the analogous verification with the current numerical bounds of [1] and confirms a strictly positive slack. Section 9 revisits the same FMM step empirically, asking when its surrounding class-routing machinery actually fires, the structural counterpart to the algebraic feasibility we just verified.

8 Phases and the full algorithm

The restricted algorithm relies critically on the fact that A and C are frozen: the running aggregate $A^{L*} \cdot B_{<i,DD} = A^{L*} \cdot B_{<i-1,DD} + A^{L*} \cdot B_{i-1,DD}$ that drives the chunk machinery only makes sense when the left factor does not change. Once A also receives updates, this telescoping breaks down and the FMM step has to be redesigned.

The full algorithm of [3] replaces chunks with *phases* of size $m^{1-\delta}$ for a constant $\delta > 0$. The mechanism splits each phase into three concurrent activities, all sharing the per-update budget of $m^{2/3-\varepsilon}$:

Slow path during the phase. Queries are answered using the precomputed products of the *previous* phase, combined with on-the-fly evaluation against the updates accumulated so far in the current phase. Each such query costs $O(m^{2/3-\varepsilon})$ in the worst case.

FMM precomputation in parallel. During the $m^{1-\delta}$ updates of the current phase, the algorithm amortises a single multiplication of two square matrices of dimension $m^{2/3+2\varepsilon}$ across the phase. The per-update share is $(m^{2/3+2\varepsilon})^\omega / m^{1-\delta}$ time, which the constraint below forces to be $O(m^{2/3-\varepsilon})$.

Roll-forward at phase boundaries. When the phase ends, the freshly computed products become the new “previous-phase” data structure, and a new phase begins.

Because the per-update cost of every concurrent activity is $O(m^{2/3-\varepsilon})$ on every update, not merely on average, the resulting bound is

worst-case. The cost analysis in [3, §4] distills to the single inequality

$$1 - \delta \geq (2\omega + 1)\varepsilon + (\omega - 1)\frac{2}{3},$$

which has a feasible solution with $\delta > 0$ if and only if $\omega < 5/2 - O(\varepsilon)$. This is the conceptual content of the result: the speedup vanishes long before the cubic barrier, and Strassen’s $\omega \approx 2.81$ [15] is therefore *not* sufficient. Only the asymptotically aggressive bounds of [1], which give $\omega \approx 2.371339$, push the exponent below $5/2$ at all. At the conjectural $\omega = 2$ with $\varepsilon = 1/24$, the inequality binds at $\delta = 1/8$ (since $1 - 1/8 = 7/8 = 5/24 + 16/24 = (2 \cdot 2 + 1)/24 + 2/3$).

Theorem 6 (Main result of [3], restated). *There is an algorithm for counting 4-cycles in any fully dynamic graph in worst-case update time $O(m^{2/3-\varepsilon})$ for some constant $\varepsilon > 0$. With current $\omega = 2.371339$, $\varepsilon \approx 0.0098$; with $\omega = 2$, $\varepsilon = 1/24$.*

We restate Theorem 6 verbatim from [3] and do *not* reprove it. Its proof combines the restricted algorithm of this section, the phase machinery sketched above, the removal of Assumption 1 via tiny-vertex handling [3, §6], and the removal of Assumption 2 via vertex-class transitions [3, §7], none of which we reproduce. The formal contribution of this paper is limited to Theorem 1 (the general-to-layered reduction), Lemma 2 (the $O(n)$ baseline), Theorem 4 (the rectangular FMM step), and Lemma 5 (feasibility of the constraint system at $\omega = 2$).

9 Experiments

Section 7 identified the rectangular FMM step as the algebraically load-bearing component of the restricted variant; this section is the empirical complement, asking the structural counterpart of the same question for the class-routing machinery that feeds that step. The warm-up of Section 6 avoids $O(n)$ -per-update enumeration when an endpoint is high-degree by storing matrix products restricted to high-degree submatrices and dispatching by class. We ask empirically: when does this class-routing actually pay off in wall-clock time? Comparing WARMUP-V3 only to SIMPLE-WEDGE (which sees the union graph and pays $O(\deg_{\text{union}}(u))$ per update) is misleading: a layer-aware baseline already avoids much of that cost. We therefore introduce LAYERED-SIMPLE-WEDGE, the apples-to-apples baseline.

9.1 Setup

We compare three counters with a common insert/delete/count interface, all written in Python 3.11 with NumPy. SIMPLE-WEDGE is the algorithm of Section 5 as written: it maintains a dense union-graph wedge matrix W and pays $O(\deg(u) + \deg(v))$ per update. LAYERED-SIMPLE-WEDGE is the same idea, made aware of the four-layer structure: it maintains only the wedge counts $W_{13}[i, w] = \sum_j A[i, j]B[j, w]$ relevant to layered 4-cycles and answers a D -edge update by $\sum_{w \in N_C(I)} W_{13}[i, w]$. It counts exactly the same object as the warm-up ($|J| = \text{tr}(ABCD)$) and is the apples-to-apples baseline. WARMUP-V3 is a faithful-in-shape implementation of Section 6 with two simplifications, both stated in code/README.md: NumPy GEMM substitutes for fast rectangular matrix multiplication, and Assumptions 1+2 are fixed at construction so updates that would change a vertex’s class raise an error. Chunk size is fixed at one, collapsing the chunk-Dense class to empty.

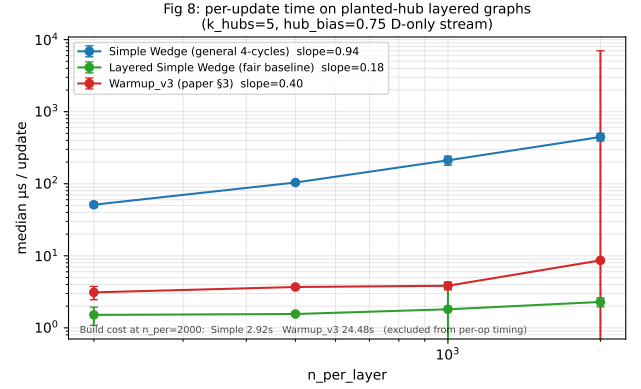


Figure 3: Planted-hub graphs ($k_{\text{hubs}} = 5$ in L_1 only). SIMPLE-WEDGE scales linearly in n (slope 0.93), the textbook $O(\deg_{\text{union}}(\text{hub}))$ worst case. The layered baseline beats WARMUP-V3 by $\approx 1.6\times$: with no hubs in L_4 , every D -update is HL or LL and the $O(1)$ HH -lookup never fires.

Inputs are layered graphs built by `layered_hub`: a *planted-hub* family with $k_{\text{hubs}} = 5$ vertices in L_1 at degree linear in n , and a *bilateral-hub* family with 5 such hubs in both L_1 and L_4 , the only setting that triggers HH -class queries in WARMUP-V3. Both run a D -only stream biased toward hub vertices (75% for planted-hub, 85% for bilateral). Per-update times are medians over 200 updates across 3 random seeds with IQR error bars; build costs are reported separately, since WARMUP-V3 pays an $O(n^3)$ GEMM precompute that should not enter a per-update headline. A correctness gate (61 lockstep tests, all passing) verifies that $\text{WARMUP-V3.COUNT}() = \text{tr}(ABCD) = \text{LAYERED-SIMPLE-WEDGE.COUNT}()$ after every step of random $B+D$ streams.

9.2 Two regimes

Hubs only on one side (Figure 3). At $n_{\text{per-layer}} = 2000$ with hubs only in L_1 : SIMPLE-WEDGE costs $\approx 420 \mu\text{s}/\text{update}$ (linear in n), WARMUP-V3 $\approx 4.1 \mu\text{s}/\text{update}$, LAYERED-SIMPLE-WEDGE $\approx 2.7 \mu\text{s}/\text{update}$; the layer-aware baseline beats the warm-up by $\approx 1.6\times$. The reason is structural: with hubs only in L_1 , every D -update query falls into the HL or LL class. The HL lookup in WARMUP-V3’s $A^{H*} \cdot B$ table iterates $|N_C(I)|$ entries, exactly the same row sum as the layered baseline, but with extra class-dispatch overhead in Python. WARMUP-V3’s distinguishing $O(1)$ HH -lookup is never triggered.

Hubs on both sides (Figure 4). Planting 5 hubs in L_4 as well pushes most D -updates into the HH cell of WARMUP-V3’s dispatch table, where the precomputed $A^{H*} \cdot B \cdot C^{*H}$ matrix yields an $O(1)$ lookup. At $n_{\text{per-layer}} = 2000$, LAYERED-SIMPLE-WEDGE costs $34.1 \mu\text{s}/\text{update}$ while WARMUP-V3 costs $6.5 \mu\text{s}/\text{update}$, a $5.2\times$ speedup, growing with n . The layered baseline must iterate $|N_C(L_4 \text{ hub})| = \Theta(n)$ entries; WARMUP-V3 reads one entry of its precomputed table. This is the mechanism promised by Section 6: the warm-up’s win comes from precomputed matrix products at high-degree submatrices, and only when the input has hubs on both sides of the cyclic ordering.

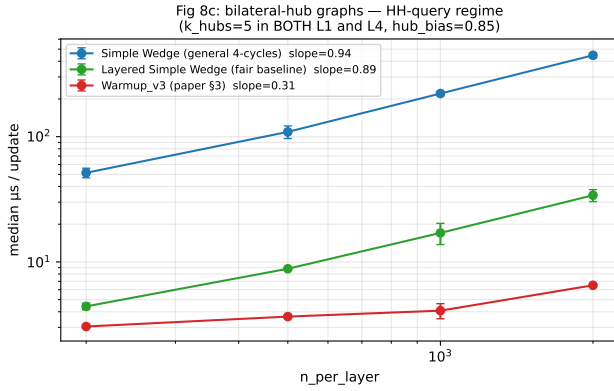


Figure 4: Bilateral-hub graphs ($k_{\text{hubs}} = 5$ in both L_1 and L_4). Most D -updates now hit the HH cell of WARMUP-V3’s dispatch table; the layered baseline still pays $O(\deg_C(L_4 \text{ hub})) \approx O(n)$. WARMUP-V3 wins, by a factor that grows with n .

9.3 Caveat

We implemented WARMUP-V3 faithfully *in shape* but with NumPy GEMM standing in for fast rectangular FMM (no practical implementation of the latter exists at our scale; even Strassen’s algorithm needs n in the thousands to break even with BLAS [1]). The $O(n^3)$ GEMM precompute is large in absolute terms: at $n_{\text{per-layer}} = 2000$ it takes 23 s, against 0.10 s for LAYERED-SIMPLE-WEDGE. Whether the warm-up’s per-update $O(m^{2/3-\epsilon_1})$ actually beats the layered baseline’s $O(m^{1/2})$ in wall-clock, with FMM in place, is the central practical question raised by [3]; our experiments cannot answer it because we do not have rectangular FMM. What our experiments *can* establish is that the warm-up’s class-routing mechanism, in isolation from its FMM ingredient, helps only on graphs whose hub structure makes HH -class queries fire frequently, which is precisely the structural assumption baked into the design of Section 6.

10 Conclusion and further work

We presented the recent algorithm of Assadi and Shah for dynamic 4-cycle counting through the lens of incremental view maintenance for the cyclic four-way join, asking one guiding question throughout: where, exactly, does the sub- $m^{2/3}$ improvement live? We surveyed the line of work culminating in their result (Section 2), recalled the layered reduction that lets us study the problem on a 4-layered graph (Section 4), presented a simple $O(n)$ wedge-maintenance baseline (Section 5), and exposed the conceptual core of the Assadi-Shah algorithm: the vertex-class partition with its precomputed FMM products (Section 6), the rectangular FMM step that drives the parameter trade-off (Section 7), and the phase mechanism that turns those precomputed products into a worst-case guarantee (Section 8). The trace isolates a single pressure point: the rectangular FMM product $A^{L^*} \cdot B_{i,DD}$, the only step in the entire pipeline whose cost is governed by ω .

Our two formal contributions probe this pressure point from complementary directions. Lemma 5 is a closed-form algebraic certificate that the constraint system at the heart of the restricted

algorithm is feasible at the conjectural optimum $\omega = 2$, with explicit rational parameters $\epsilon_1 = 1/24$, $\epsilon_2 = 5/24$, and $\epsilon = 1/24$. Where the source paper verifies feasibility numerically with the current rectangular FMM bounds (Appendix B of [3]), the $\omega = 2$ closed form makes interpretable why the rectangular FMM constraint binds tight (the only ω -dependent constraint of the five that does), identifying FMM as the load-bearing ingredient of the parameter trade-off. The supporting dimension analysis (Theorem 4) is taken from the source paper. In Section 9 we located, empirically, the input regime in which the warm-up’s class-routing actually pays off (graphs with hubs on both sides of the cyclic ordering), by contrasting it with a layer-aware baseline that already captures most of the warm-up’s benefit on simpler inputs. The two angles agree: the algebraic improvement of [3] lives at the rectangular FMM step, and the surrounding class-routing machinery exists to feed that step the well-conditioned inputs (bilateral hubs) that justify its bookkeeping cost.

We highlight three directions for further work. First, the gap between the $O(m^{2/3-\epsilon})$ upper bound of [3] and the $\Omega(m^{1/2-\gamma})$ OMv-conditional lower bound of [7]: where on $[1/2, 2/3]$ does the truth lie? The OMv reduction passes through boolean matrix-vector products and so does not rule out further FMM-based speedups, while [3] explicitly notes that a combinatorial $\Omega(m^{2/3})$ lower bound is not yet ruled out either, so the gap could close from either side. Second, IVM-with-FMM as a research direction: this is the first time fast matrix multiplication has been leveraged for join-size estimation in dynamic graphs, and the authors suggest the technique could inspire a new class of IVM algorithms [3]. Where else does FMM break a “natural” barrier in dynamic IVM? Other cyclic joins, longer cycles, richer pattern queries? An adjacent direction is approximate counting under updates: [16] combines sampling with FMM for static approximate triangle counting, and a dynamic counterpart would close a parallel gap to the one studied here. Third, on the empirical side, a faithful implementation of the restricted algorithm with a competitive rectangular FMM routine would test whether the constants of [1] let the asymptotic improvement surface at realistic graph sizes; our benchmarks of the simple algorithm cannot speak to this, and we view it as the natural next step.

References

- [1] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. 2025. More Asymmetry Yields Faster Matrix Multiplication. In *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2005–2039.
- [2] Noga Alon, Raphael Yuster, and Uri Zwick. 1997. Finding and Counting Given Length Cycles. *Algorithmica* 17, 3 (1997), 209–223.
- [3] Sepehr Assadi and Vihan Shah. 2025. An Improved Fully Dynamic Algorithm for Counting 4-Cycles in General Graphs using Fast Matrix Multiplication. arXiv:2504.10748 [cs.DS] PODS 2025, Article 091. <https://doi.org/10.1145/3725228>.
- [4] David Eppstein, Michael T. Goodrich, Darren Strash, and Lowell Trott. 2012. Extended Dynamic Subgraph Statistics Using h-Index Parameterized Data Structures. *Theoretical Computer Science* 447 (2012), 44–52.
- [5] David Eppstein and Emma S. Spiro. 2009. The h-Index of a Graph and its Application to Dynamic Subgraph Statistics. In *Algorithms and Data Structures (WADS 2009) (LNCS, Vol. 5664)*. Springer, 278–289.
- [6] Kathrin Hanauer, Monika Henzinger, and Qi Cheng Hua. 2022. Fully Dynamic Four-Vertex Subgraph Counting. In *1st Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2022) (LIPIcs, Vol. 221)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 18:1–18:17.
- [7] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. 2015. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the*

- Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*. 21–30.
- [8] Xiao Hu, Qizhi Liu, and Stavros Sintos. 2022. Lower Bounds for Dynamic Maintenance of Cyclic Joins. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. 275–286.
 - [9] Ahmet Kara, Hung Q. Ngo, Milos Nikolic, Dan Olteanu, and Haozhe Zhang. 2020. Maintaining Triangle Queries under Updates. *ACM Transactions on Database Systems* 45, 3 (2020), 11:1–11:46.
 - [10] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network Motifs: Simple Building Blocks of Complex Networks. *Science* 298, 5594 (2002), 824–827.
 - [11] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2018. Worst-Case Optimal Join Algorithms. *J. ACM* 65, 3 (2018).
 - [12] Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2014. Skew Strikes Back: New Developments in the Theory of Join Algorithms. *ACM SIGMOD Record* 42, 4 (2014), 5–16.
 - [13] Pedro Ribeiro, Pedro Paredes, Miguel E. P. Silva, David Aparicio, and Fernando Silva. 2021. A Survey on Subgraph Counting: Concepts, Algorithms, and Applications to Network Motifs and Graphlets. *Comput. Surveys* 54, 2 (2021), 1–36.
 - [14] Siddhartha Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. 2020. The Ubiquity of Large Graphs and Surprising Challenges of Graph Processing: Extended Survey. *The VLDB Journal* 29 (2020), 595–618.
 - [15] Volker Strassen. 1969. Gaussian Elimination is Not Optimal. *Numer. Math.* 13, 4 (1969), 354–356.
 - [16] Jakub Tětek. 2022. Approximate Triangle Counting via Sampling and Fast Matrix Multiplication. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022) (LIPIcs, Vol. 229)*. 107:1–107:20.